

BOOT4DIM

부트사차원

C언어 중급 교육

PRESENTER : 최민혁

2회차

2026. 3. 19 (목요일)



CONTENTS

구조체와 사용자 정의 자료형

01

구조체의 의의



04

Linked List



02

구조체의 정의와 Typedef



05

총정리



03

구조체 포인터와 `->` 연산자



06



01

구조체의 의미



구조체: 서로 다른 종류의 데이터들을 하나의 바구니에 담아 새로운 자료형으로 정의

```
char name[10];  
int math;  
int english;  
float average;
```

한 요소에 존재하는 변수들을
통제하기 어려움

```
struct info {  
    char name[10];  
    int math;  
    int english;  
    float average;  
};
```

변수들을 통제하기 쉬워짐

01

구조체의 의의



구조체: 서로 다른 종류의 데이터들을 하나의 바구니에 담아 새로운 자료형으로 정의

```
int main(){
    char name1[10] = "hong";
    int math1 = 92;
    int english1 = 86;
    float average1 = (math1 + english1) / 2;

    char name2[10] = "choi";
    int math2 = 88;
    int english2 = 90;
    average2 = (math2 + english2) / 2;

    return 0;
}
```

만약 학생이 20명이 넘는다면...?

```
struct info {
    char name[10];
    int math;
    int english;
    float average;
};

int main(){
    struct info student1;
    struct info student2;
}
```

한 번에 선언이 가능하다.



```
#include <stdio.h>

struct info{
    char* name;
    int math;
    int english;
    float average;
};

int main(){
    struct info student1;
    struct info student2;

    student1.name = "Bill";
    student1.math = 98;
    student1.english = 79;
    student1.average = (student1.math + student1.english) / 2.0;
}
```

1. Main함수 이전 원하는 구조체를 선언한다.
2. (선언 종료 후 ;)

2. 지역변수 선언
(‘info’라는 구조체임을 알리기 위해 struct 붙이기)

3. ‘.’ 연산자를 이용하여 구조체의 요소에 접근

4. ‘.’ 연산자를 이용하면 초기화, 참조 모두 가능하다.

02

구조체의 정의와 Typedef



Typedef: 기존 자료형에 새로운 별명을 붙임



```
typedef [기존 타입] [새로운 이름];
```



```
typedef unsigned int uint_t;
```

Tip: 새로운 자료형을 만들 때는 _t를 사용하자.
코드가 길 때 가독성이 올라간다. (type의 줄임말)



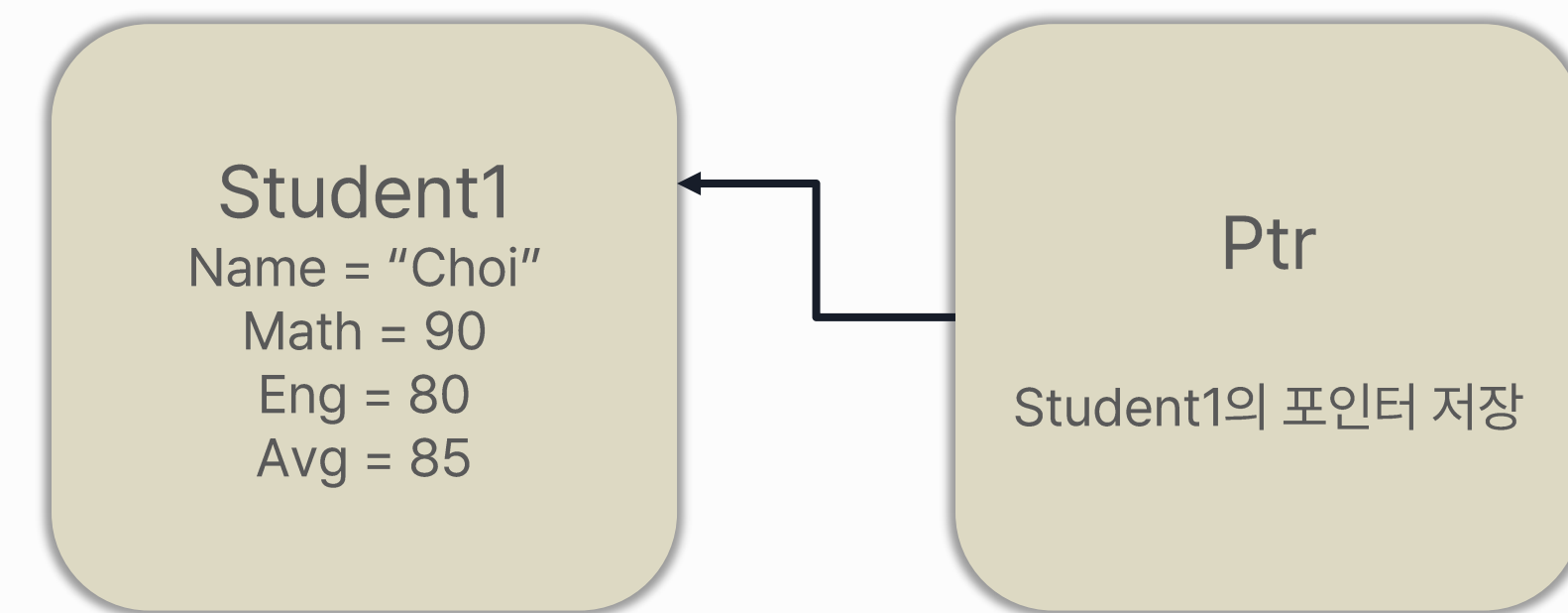
```
typedef struct info{  
    char name[10];  
    int math;  
    int english;  
    float average;  
} info_t;  
  
int main(){  
    info_t student1;  
    info_t student2;  
    ...  
}
```




```
typedef struct info{
    char name[10];
    int math;
    int english;
    float average;
} info_t;

int main(){
    info_t student1 = {"Choi", 90, 80, 85};
    info_t* ptr = &student1;

    printf("%p", ptr);
    printf("%p", &student1);
}
```

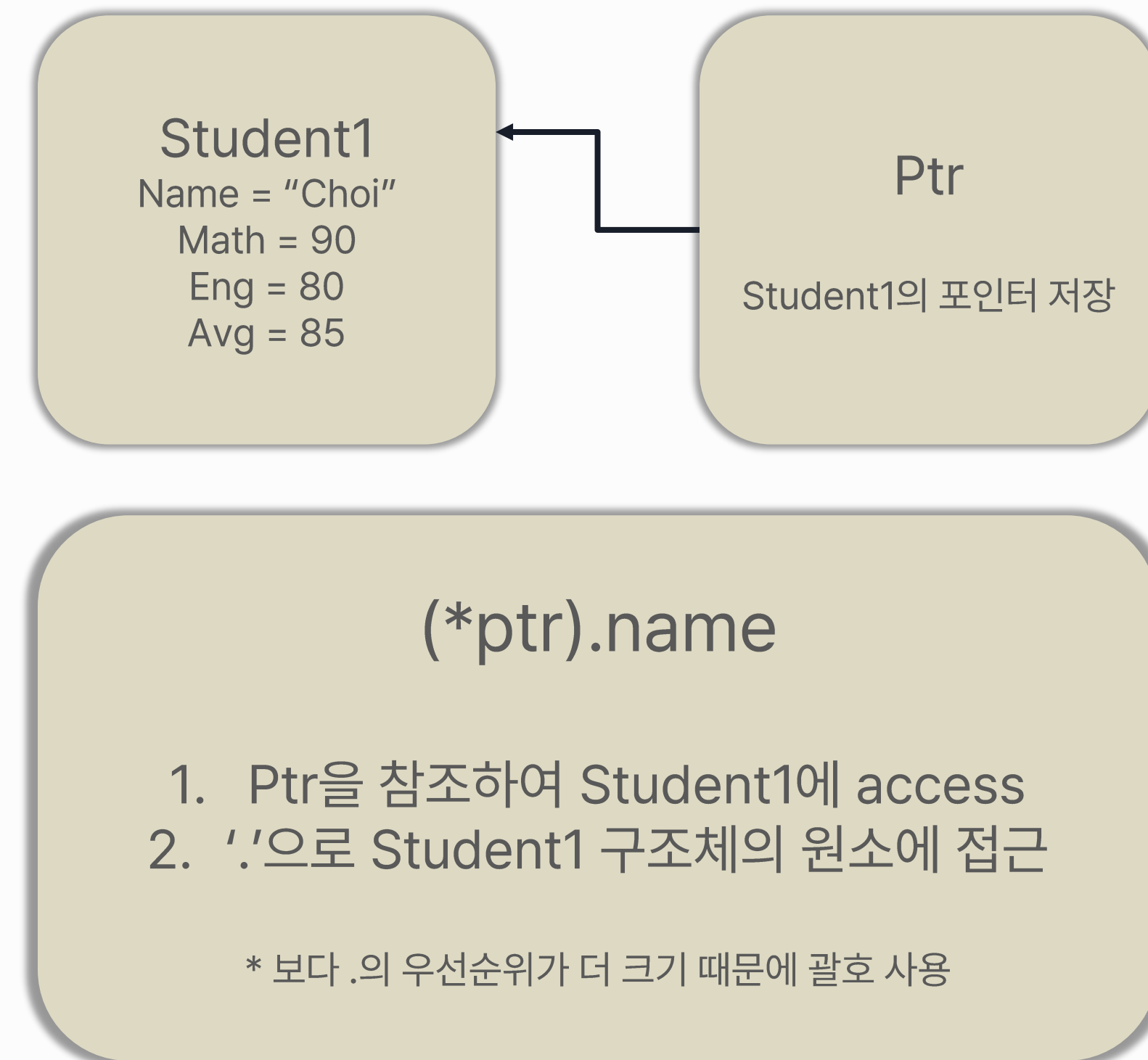




```
typedef struct info{
    char name[10];
    int math;
    int english;
    float average;
} info_t;

int main(){
    info_t student1 = {"Choi", 90, 80, 85};
    info_t* ptr = &student1;

    printf("%s", (*ptr).name);
    printf("%d", (*ptr).math);
    printf("%d", (*ptr).english);
    printf("%f", (*ptr).average);
}
```



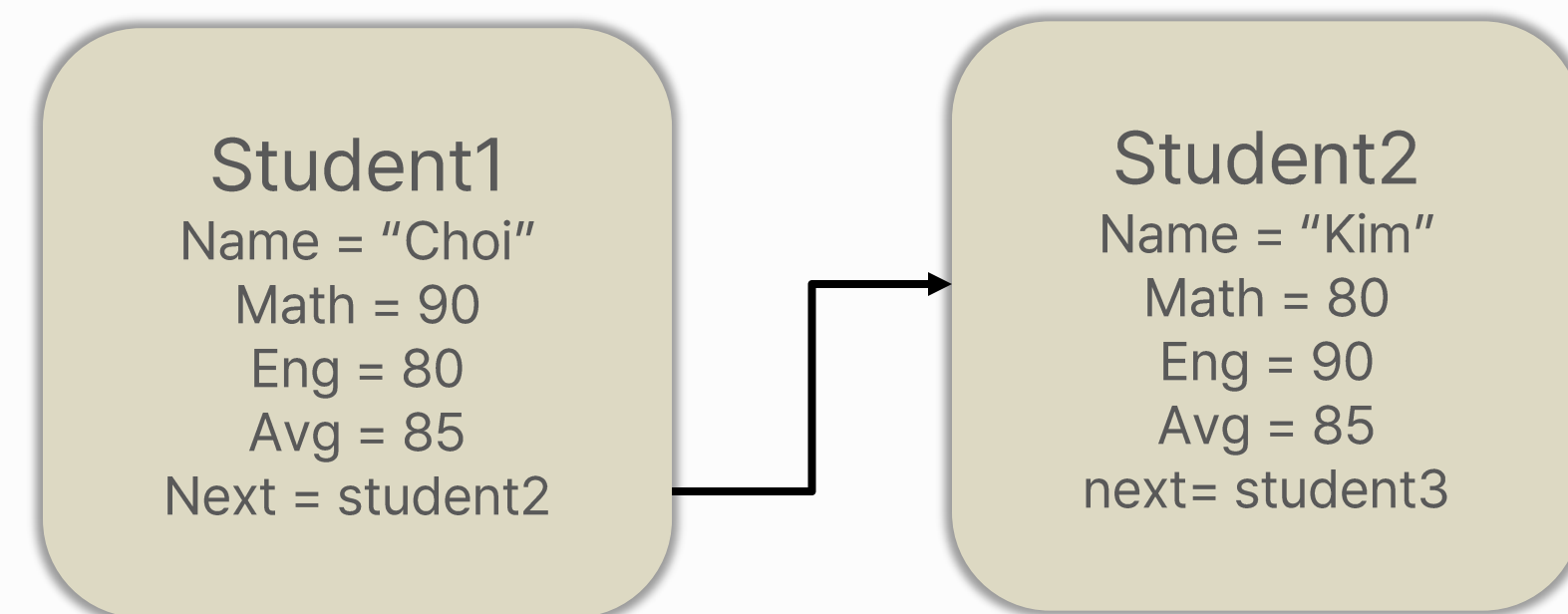


```
typedef struct info{
    char name[10];
    int math;
    int english;
    float average;
    info_t* next;
} info_t;

int main(){
    info_t student1, student2, student3....
    info_t* ptr = student1;

    ...

    ((*ptr).next).math;
}
```



구조체 포인터를 이용하여 다른 구조체를 참조할 때

*(). 의 형태를 많이 쓰게 된다.

이를 방지하기 위해 -> 연산자를 사용한다.



->와 *().
동일한 표현이다

Call By Value

값에 의한 호출

호출자의 인자, 수신자의 매개변수는 다른 변수

함수 내에서의 매개변수의 변화는 원본 변수에 반영되지 않음

```
typedef struct info{
    char name[10];
    int math;
    int english;
    float average;
} info_t;

int main(){
    info_t student1 = {"Choi", 90, 80, 85};
    info_t* ptr = &student1;

    printf("%d", ptr->math);
    printf("%d", ptr->english);

    printf("%d", (*ptr).math);
    printf("%d", (*ptr).english);
}
```

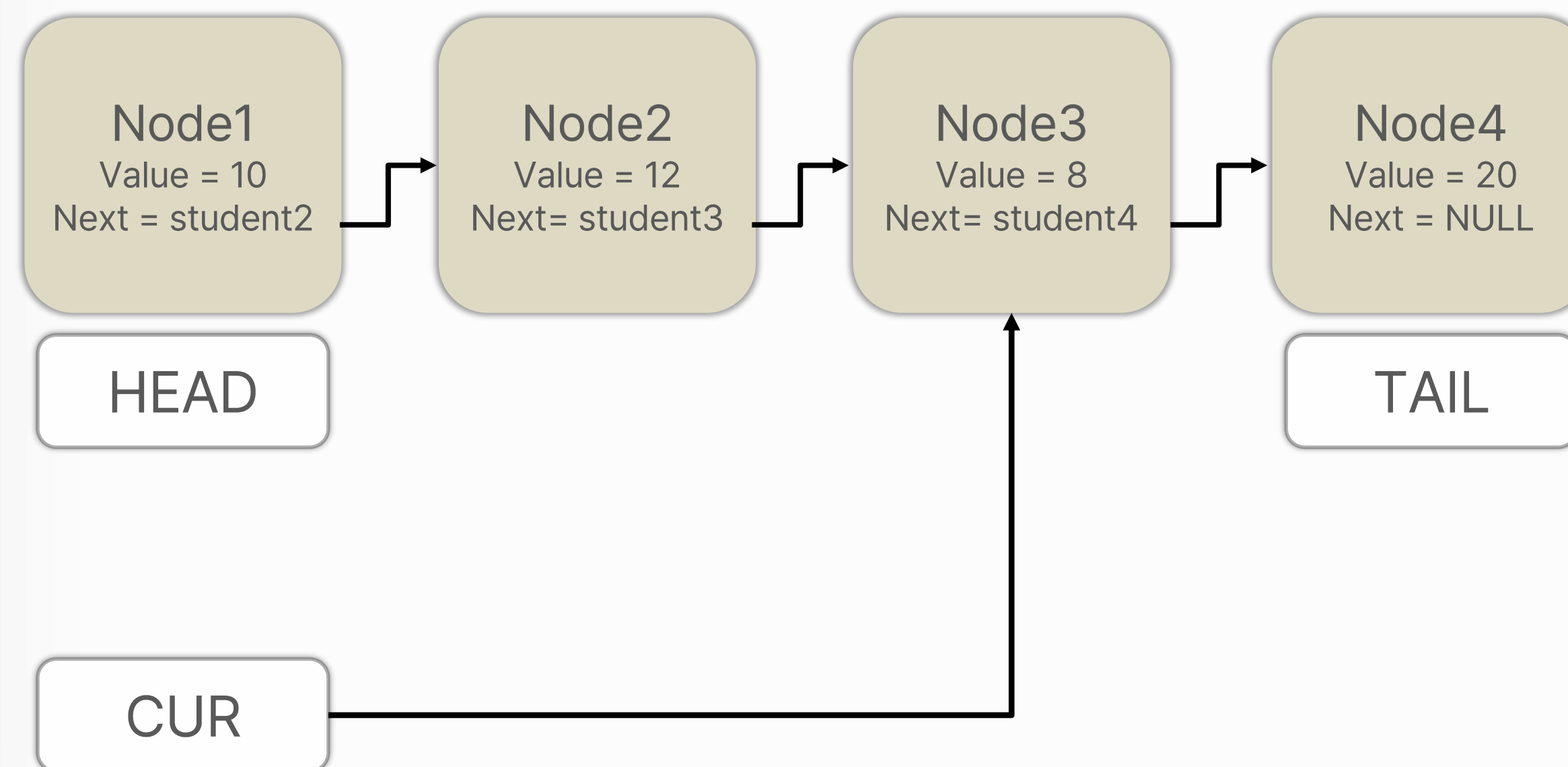
Linked List

구조체를 사용한 단방향 링크드 리스트 구현

구조체를 선언하고, 원하는 자료와 함께 해당 구조체의 포인터 하나 지정한다.

구조체 포인터 HEAD, TAIL을 지정하여 연결된 리스트들의 처음과 끝을 명시한다.

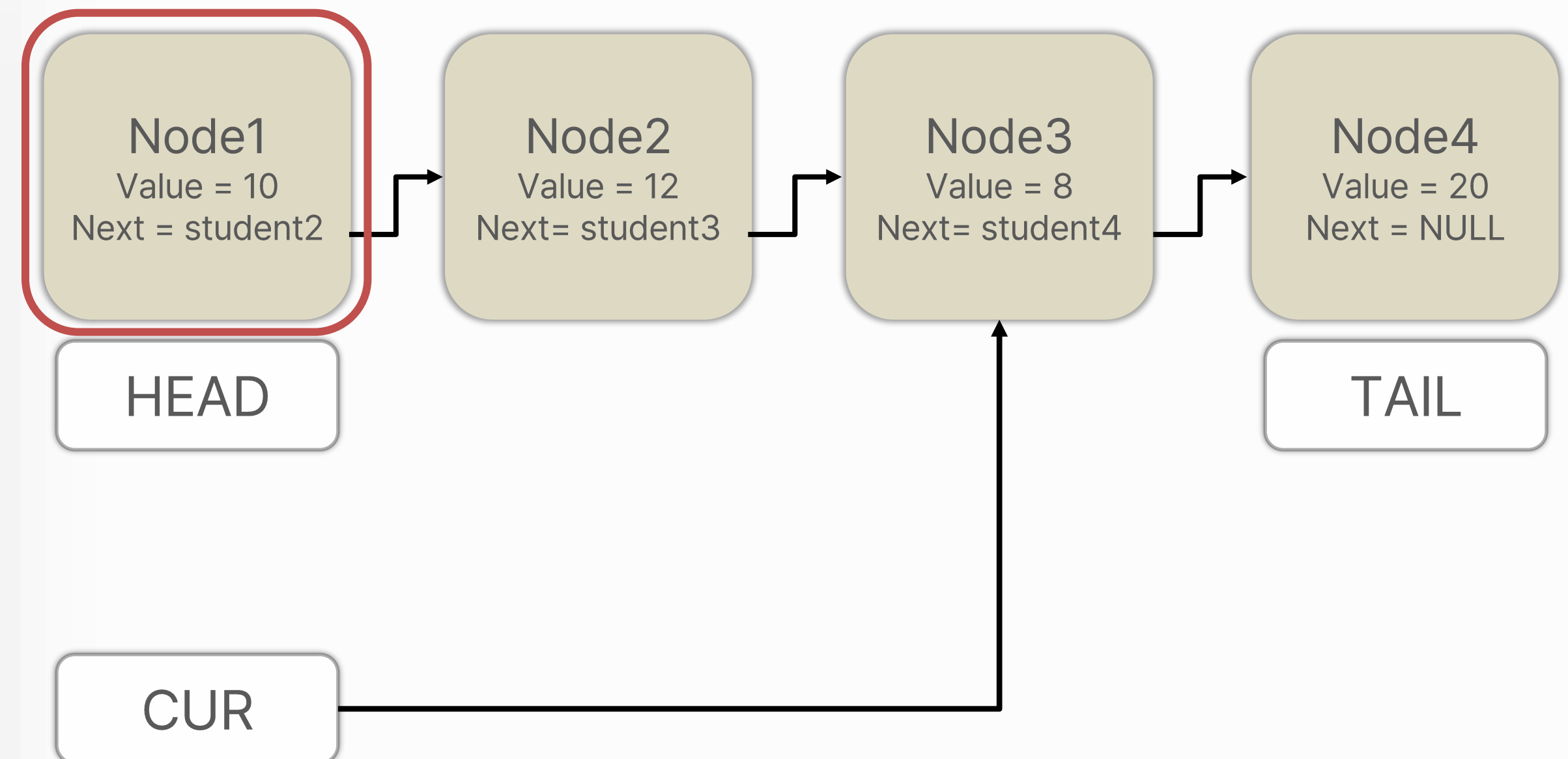
구조체 포인터를 사용하여 리스트 사이를 넘나들어 원하는 구조체에 접근, 리스트 추가, 삭제를 시행한다.



Linked List

1. 구조체를 선언하여 원하는 자료와, 다음 노드를 가리킬 포인터를 선언

```
typedef struct Node{  
    int value;  
    Node* next;  
} Node;
```



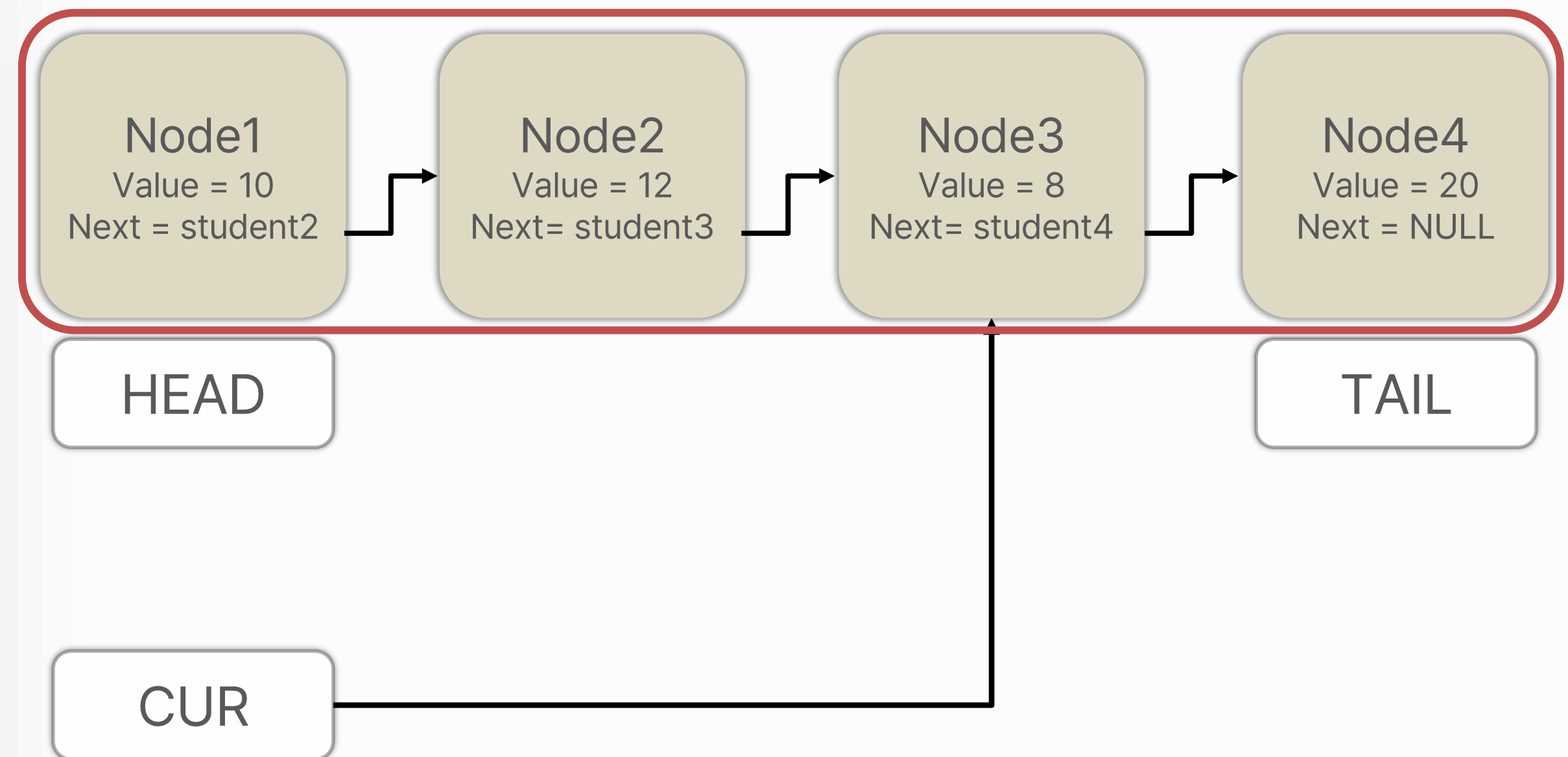
Linked List

2. 노드 생성 로직을 만든다.

이 때 포인터는 다음 노드의 포인터값을 저장해야 한다.



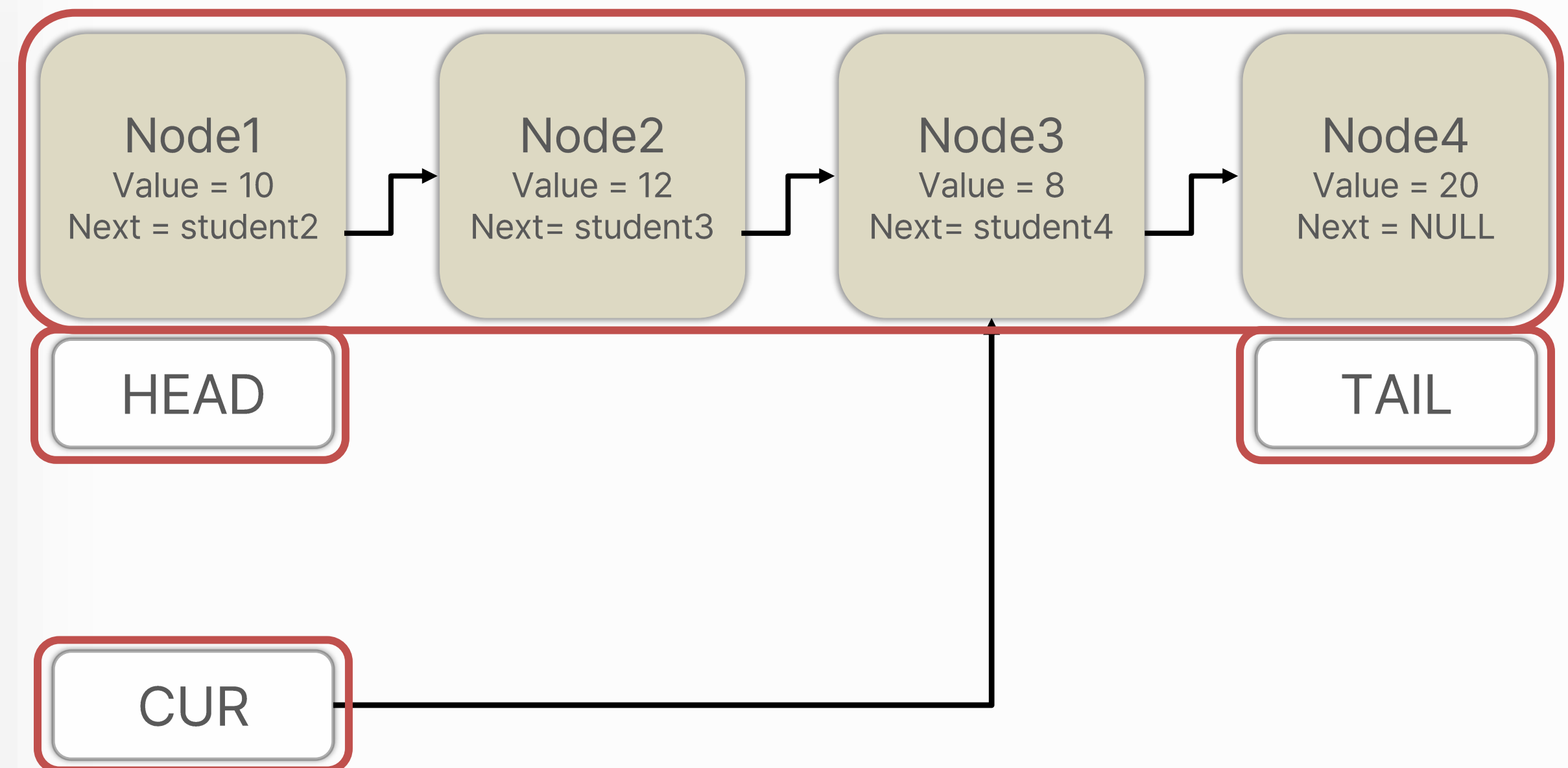
```
int main() {  
    Node Node1, Node2, Node3, Node4;  
    Node1 = {10, &Node2};  
    Node2 = {12, &Node3};  
    Node3 = {8, &Node4};  
    Node4 = {20, NULL};  
}
```



Linked List

3. 리스트의 처음과 끝 명시를 위한
HEAD, TAIL 을 선언한다.
각 노드에 접근하기 위한 CUR을 선언한다.

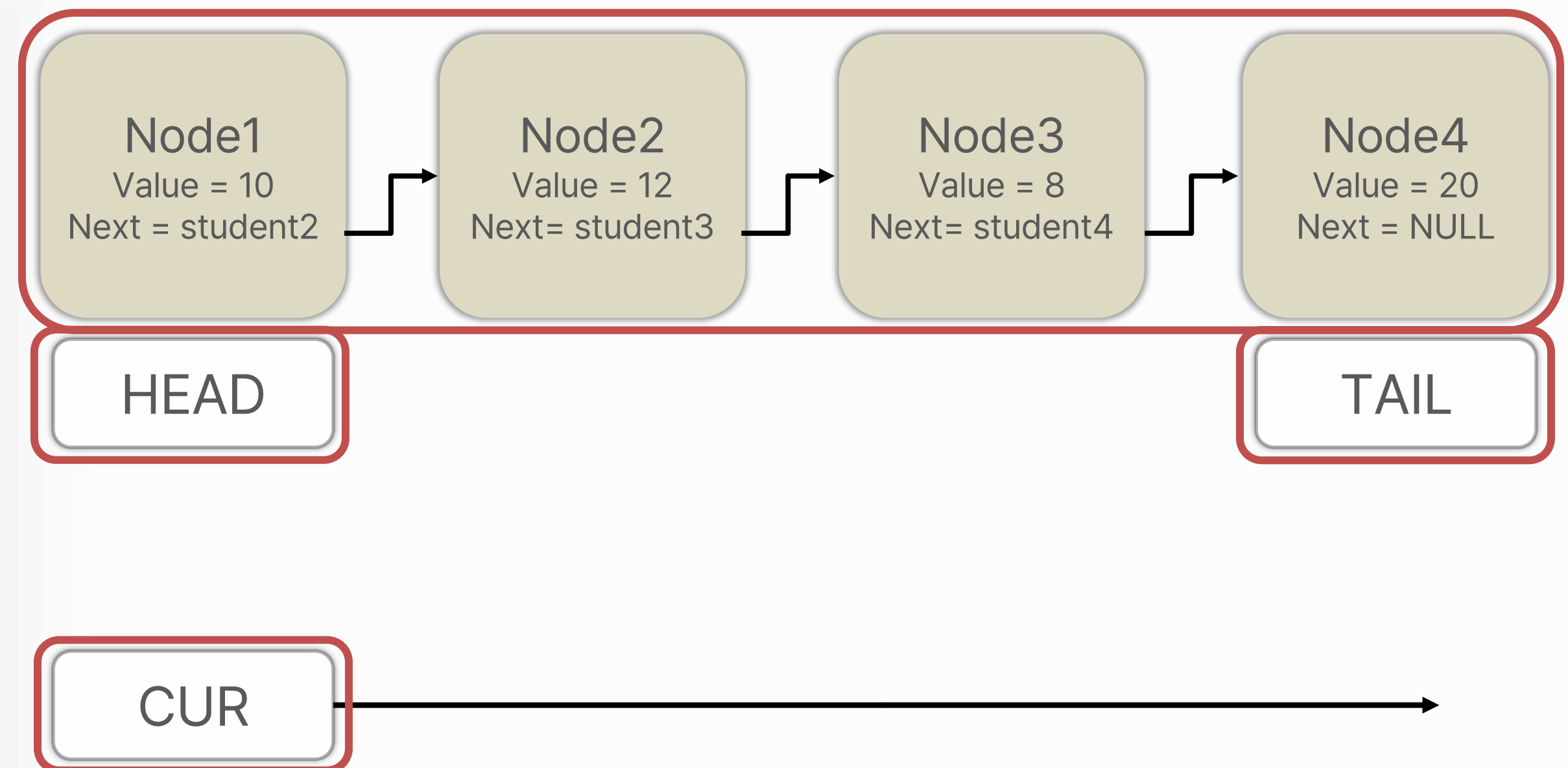
```
int main() {  
    Node* HEAD = &Node1;  
    Node* TAIL = &Node4;  
  
    Node* CUR = &Node1;  
}
```



Linked List

4. CUR을 움직이며 각 노드의 Value를 참조한다.
이때 다음 Node로의 움직임은 현재 Node의
다음 요소의 포인터를 사용한다.

```
int main() {  
    while(CUR != NULL){  
        printf("%d", CUR->value);  
  
        CUR = CUR->next;  
    }  
}
```





```
#include <stdio.h>

// TASK1. item_t 구조체 정의하기
// (멤버: int id, float price, struct item* next)

int main() {
    item_t i1, i2, i3;
    //TASK2 : 각각의 원소값 초기화하기

    //TASK3 : item_t 포인터 변수 HEAD, TAIL, CUR을 이용하여 적절히 초기화하기

    //TASK4 : while문 사용하여 모든 노드의 id, price값 출력하기

    return 0;
}
```

[실습 문제] 링크드 리스트를 이용한 가격표

Task1: typedef로 item_t 구조체 정의하기
구조체 멤버: int id, float price, struct item* next

Task2: 지역변수로 선언된 i1, i2, i3의 값 초기화
i1: ID: 1, price: 8.3 / i2: ID:2, price: 10.1/ i3: ID: 3, price: 17.0

Task3: item_t 포인터 변수 HEAD, TAIL, CUR
초기화하기

Task4: while문으로 모든 노드의 id, price값 출력



Q & A